COP4520 Project 2 Report

Felix Hoang | NetID: phuchoang

Optimized CUDA techniques for histogram computation

The CPU reference remains the single-threaded brute-force SDH algorithm from Project 1. Project 2 optimizes the GPU implementation, rather than changing the CPU algorithm. All versions use the same double-precision coordinates, BOX_SIZE = 23000, and srand(1). Every unordered pair is counted once, and the histogram uses 64-bit counts.

Utilizing high-speed shared memory

A two-dimensional grid divides the point pairs into tile pairs. Each thread retains one point in registers, while neighboring threads cooperatively load the second tile into shared memory. The block reuses that tile for its distance calculations, reducing repeated global-memory reads. This goes beyond Project 1, which keeps the output histogram in shared memory but repeatedly reads the input points from global memory.

Ensuring safe concurrent updates

The kernel uses atomicAdd to avoid lost updates. When the histogram fits, each block accumulates into its own shared-memory histogram, synchronizes, and adds its nonzero counts to the global histogram. The program checks the GPU shared-memory limit after reserving the point tile. If the histogram is too large, the same tiled kernel updates the global histogram directly. This fallback satisfies the Project 2 requirement to support histograms larger than per-block shared memory.

Global memory access and workload distribution

Threads load neighboring point records and reuse the shared tile. The retained array of atom structs has strided coordinate accesses across threads, so this implementation does not claim fully coalesced coordinate loads. Separate coordinate arrays could improve this, but were omitted to keep the change small. Tile-pair scheduling distributes the triangular pair space more evenly than assigning a complete remaining row to each thread. Checks for j > i and the final partial tile prevent duplicate pairs and invalid accesses. Rounded double multiply/add operations preserve CPU agreement near bucket boundaries.

Overview

The implementation combines input tiling, register reuse, and block-local histogram privatization in one kernel. The supplied block size is used directly. CUDA events surround the kernel launch, and the stop event is synchronized before reading the elapsed time. Allocation, histogram clearing, CPU computation, transfers, and printing are excluded from GPU kernel timing. Valid program output contains only the CPU histogram, GPU histogram, differences, and total optimized kernel execution time.

Comparative analysis of spatial distance histogram

Matching the sample report, the main comparison uses **bucket width 256 and block size 128**, with 10,000, 100,000, and 500,000 points. Measurements were collected on GAIVI using a GeForce GTX 1080 Ti and CUDA 11.4.48. The separate benchmark includes the actual Project 1 and Project 2 source files without changing their kernels. All GPU versions are measured with synchronized CUDA events using the same input and compiler options. GPU entries are means of three runs; CPU entries are one timed call to the original CPU baseline, excluding setup and output.

Points	CPU (s)	P1 global (ms)	P1 shared (ms)	P2 tiled (ms)
10,000	0.981	23.745	14.458	8.831
100,000	98.112	1795.755	634.972	687.382
500,000	N/A	44902.118	16034.479	17571.442

Points	CPU / Project 2	P1 global / P2	P1 shared / P2
10,000	111.14x	2.69x	1.64x
100,000	142.73x	2.61x	0.92x
500,000	N/A	2.56x	0.91x

Each speedup is the baseline computation time divided by the Project 2 kernel time. Values below 1 mean Project 2 was slower. This compares computation only, not end-to-end application time. The 500,000-point CPU run is omitted, matching the sample report's N/A entry; no CPU time is estimated. These are new measurements, not copied sample timings.

At 10,000 points, Project 2 was 1.64x faster than the Project 1 shared-memory kernel. At 100,000 points, Project 2 took 687.382 ms versus 634.972 ms for Project 1 shared memory, so tiling did not improve this case. Extra tile blocks, synchronization, and histogram initialization can offset input reuse; no profiler-based attribution was performed. The results demonstrate that the benefit depends on problem size rather than guaranteeing that every added optimization is faster.

Correctness and additional experiments

At 10,000 and 100,000 points, every timed GPU run matched the CPU in every bucket. For 500,000 points, the CPU baseline was not run: all GPU versions agreed with the first Project 1 global-memory result and had the expected 124,999,750,000 total pairs. This larger case is GPU cross-validation, not a full CPU comparison.

The earlier validation job 557880 passed eight valid cases, including 1 and 2 points, a 257-point partial block with 96 threads, and multiple block sizes. Every bucket matched the CPU. Eight invalid-input cases were rejected. At width 5 and block size 256, 7,968 histogram buckets require 63,744 bytes, exceeding the 49,152-byte per-block shared-memory limit even before the point tile. The global-histogram fallback passed this case. Single-run times at 10,000 points and width 500 were 8.869, 9.685, and 10.541 ms for block sizes 128, 256, and 512. These supplementary measurements use different settings from the main comparison.

The program compiled and ran successfully using the course run script.